

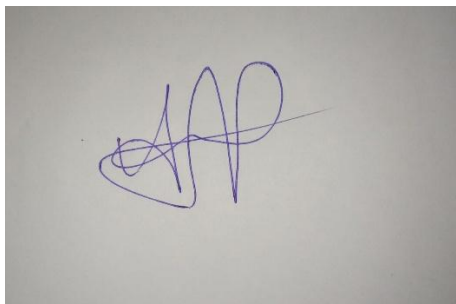
Name: Mohammed Arshad R
Register No.: 192521126
Subject: Cryptography and Network Security

Annexure A
CASE STUDY

Code of Conduct:

I **Mohammed Arshad R/192521126**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment .I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1: Weak Password Hashing in Web Application

Case Scenario

A web application stores user passwords using unsalted SHA-1 hashes. After a data breach, attackers quickly crack multiple passwords using precomputed tables. Analyze how the lack of salting and the use of SHA-1 contributed to this vulnerability, and recommend a secure password hashing strategy, justifying the choice.

1.1 Introduction

Password storage is one of the most security-critical components of any web application, because a single database breach can expose credentials for every registered user. The case described here represents a textbook password-storage failure: passwords were hashed with SHA-1 and no per-user salt was applied. This report analyses the two independent weaknesses in this design — the choice of hash algorithm and the absence of salting — explains precisely how each contributed to the rapid mass compromise of passwords after the breach, and proposes a secure replacement strategy with justification.

1.2 Root Cause Analysis

1.2.1 Weakness 1: Absence of Salting

A salt is a random, unique value added to each password before hashing. Without it, identical passwords across different user accounts always produce identical hash values, since $\text{SHA-1}(\text{password})$ is a deterministic function of the password alone. This has two severe consequences in a breach:

- **Rainbow-table attacks:** Attackers maintain enormous precomputed lookup tables that map SHA-1 hash outputs back to common passwords, dictionary words, and known-breach password lists (e.g., “rockyou.txt”, which contains over 14 million real-world passwords). Because no salt was used, the stolen hash database could be checked directly against these public tables without any additional computation, allowing an attacker to “reverse” tens of thousands of hashes within minutes.
- **Duplicate exposure:** Any two users who happened to choose the same password (a very common occurrence — password reuse and predictable choices such as “123456” or “password1” are extremely widespread) had identical hash values. Once the attacker cracked one such hash, every other account sharing that hash was compromised simultaneously at zero additional cost.

1.2.2 Weakness 2: Use of SHA-1

SHA-1 compounds the problem for two separate reasons, one cryptographic and one performance-related:

- **Cryptographic weakness:** SHA-1 has known collision vulnerabilities (demonstrated practically by the “SHAttered” attack in 2017) and has been formally

deprecated by NIST for security-sensitive use since 2011. While collision resistance is not the primary property needed for password hashing (preimage resistance is), the algorithm's broader weakening and obsolescence reflect a general erosion of the security margin that responsible systems should no longer rely on.

- **Designed-for-speed problem:** Far more critically for password storage, SHA-1 (like SHA-256 and MD5) is a fast, general-purpose hash function designed for high throughput in applications such as file integrity checks and digital signatures. This property, desirable elsewhere, is precisely wrong for password hashing. Modern GPUs and dedicated ASIC/FPGA hardware can compute several billion SHA-1 hashes per second. This means an attacker performing an offline brute-force or dictionary attack against the stolen hash file — not even relying on rainbow tables — can test enormous numbers of password guesses per second per device, making even moderately complex passwords crackable within hours to days.

1.3 How the Two Weaknesses Combined

The breach outcome described — attackers cracking multiple passwords quickly using precomputed tables — is the direct, compounding result of both flaws acting together. The missing salt made precomputed/rainbow-table attacks possible in the first place (a salted hash cannot be looked up in a generic precomputed table because the table would need to account for every possible salt value). The use of a fast hash function like SHA-1 then meant that even for the (presumably smaller) set of passwords not already present in public rainbow tables, a direct brute-force/dictionary attack against the stolen file was still fast enough to succeed quickly. In other words, salting alone would have defeated the rainbow-table shortcut, and a slow key-derivation function alone would have made brute-force impractical — but with neither protection present, both attack paths were wide open simultaneously.

1.4 Recommended Secure Password Hashing Strategy

The recommended remediation is to migrate password storage to a purpose-built, memory-hard password hashing algorithm — specifically **Argon2id** (the winner of the 2015 Password Hashing Competition and currently recommended by OWASP as the first choice), with bcrypt or PBKDF2-HMAC-SHA256 as acceptable fallbacks where Argon2 is unavailable in the platform's cryptographic library.

Requirement	Design Fix	Why It Solves the Problem
Defeat rainbow tables	Generate a unique cryptographically random salt (≥ 16 bytes) per user, stored alongside the hash	Identical passwords now yield different hashes per account; a single precomputed table can no longer be reused across the database
Requirement	Design Fix	Why It Solves the Problem

Slow down brute force	Use Argon2id with tuned cost parameters (e.g., memory=64–256 MB, iterations=2–3, parallelism=1–4)	Each guess becomes computationally and memory expensive, sharply reducing attacker throughput
Resist GPU/ASIC acceleration	Argon2id's memory-hardness forces large RAM usage per hash attempt	GPUs/ASICs have limited per-core memory, so parallel cracking scales far worse than with fast hashes like SHA-1
Safe verification	Compare computed vs. stored hash using a constant-time comparison routine	Prevents timing side-channel attacks during login
Future-proofing	Store the algorithm name and cost parameters with each hash; support transparent re-hashing on next login when parameters are upgraded	Allows security parameters to be strengthened over time without forcing a mass password reset

Table 1: Design elements of the recommended secure password hashing strategy.

1.5 Illustrative Implementation (Python, Argon2id)

```

from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError

# Cost parameters chosen to target ~250–500 ms per hash on production hardware
ph = PasswordHasher(
    time_cost=3,          # number of iterations
    memory_cost=65536,    # 64 MB of RAM per hash attempt
    parallelism=2,        # degree of parallelism
    hash_len=32,          # salt_len=16
    salt_len=16           # random salt generated internally per call
)

def store_password(password: str) -> str:
    """Returns a self-contained encoded hash string:
       algorithm, version, cost parameters, salt and hash are all embedded,
       e.g. $argon2id$v=19$m=65536,t=3,p=2$<salt>$<hash>"""
    return ph.hash(password)

def verify_password(stored_hash: str, password: str) -> bool:
    try:
        ph.verify(stored_hash, password)          # Optionally: if
        ph.check_needs_rehash(stored_hash), rehash and          # update the stored
        record so cost parameters can be upgraded later.         return True
    except VerifyMismatchError:
        return False

# --- Demonstration ---
record = store_password("Summer@2026!")
print("Stored record:", record)
print("Correct password verifies:", verify_password(record, "Summer@2026!"))
print("Wrong password verifies :", verify_password(record, "guessedpass"))

```

The encoded output string produced by the library already embeds the algorithm identifier, version, cost parameters, and a freshly generated random salt for every call, which removes

the risk of a developer accidentally reusing a salt or omitting one, a common implementation mistake in hand-rolled schemes.

1.6 Justification of the Choice

- Purpose-built for passwords: Unlike SHA-1 or SHA-256, Argon2id is explicitly designed to be slow and resource-intensive, which is the correct property for password hashing (the opposite of what is desirable for general-purpose hashing).
- Memory-hardness: Argon2id requires a configurable amount of RAM per hash computation, which severely limits the degree of parallel cracking achievable on GPUs and ASICs, unlike PBKDF2 or bcrypt which are comparatively memory-light and thus more GPU-friendly.
- Side-channel resistance: The “id” variant combines data-independent memory access (resisting cache-timing attacks) for part of its processing with data-dependent access for the remainder, giving a balanced defence suitable for password hashing in a typical web server environment.
- Industry endorsement: Argon2id is recommended as the first choice by OWASP's Password Storage Cheat Sheet and is the winner of the Password Hashing Competition, giving strong assurance of independent expert scrutiny.
- Tunable and future-proof: Its time, memory, and parallelism cost parameters can be increased over time as hardware improves, without changing the algorithm itself, extending the useful lifetime of the scheme.

1.7 Additional Defence-in-Depth Recommendations

- Enforce a minimum password length and check new passwords against known-breach password lists (e.g., via the Have I Been Pwned API) at registration/change time, rather than relying solely on hashing strength.
- Apply rate-limiting and account lockout / CAPTCHA on repeated failed login attempts to blunt online guessing even if offline hashes were never exposed.
- Add a server-side application-wide “pepper” (a secret key stored outside the database, e.g., in a hardware security module or secrets manager) in addition to per-user salts, so that a database-only breach does not immediately expose crackable hashes.
- Encrypt the password-hash column and restrict database access following the principle of least privilege, so a broader application compromise is required before hashes can even be exfiltrated.
- Plan and rehearse an incident-response procedure (forced password resets, user notification) to be executed immediately if a breach is ever detected, minimizing the window of exposure.

1.8 Conclusion

The breach in this case study was not caused by a single mistake but by the combination of two independent weaknesses: the absence of per-user salting, which enabled reuse of precomputed rainbow tables across the entire database, and the use of a fast, general-

purpose hash function (SHA-1), which made brute-force attacks against any remaining hashes practical at scale. Migrating to a memory-hard, purpose-built password hashing algorithm such as Argon2id, combined with unique random salts, constant-time verification, and complementary defences such as rate-limiting and breached-password checks, directly closes both attack paths and brings the system in line with current industry best practice.

Question 2: Digital Signature Compromise in Financial System

Case Scenario

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions. Evaluate how the compromise affects authentication and non-repudiation, and propose mitigation strategies to secure private keys and prevent such incidents.

2.1 Introduction

Digital signature schemes (such as RSA-PSS or ECDSA) are used in financial systems to provide two closely related but distinct security guarantees for a transaction: **authentication**, the assurance that the transaction was submitted by the legitimate account holder, and **non-repudiation**, the assurance that the signer cannot later deny having authorized the transaction, since only the holder of the private key could have produced a valid signature. Both guarantees rest entirely on one assumption: that the private signing key remains exclusively under the control of its rightful owner. This report evaluates what happens to those guarantees when that assumption is violated by private-key theft, and proposes concrete mitigation strategies.

2.2 How Digital Signatures Normally Provide These Guarantees

In a standard digital signature scheme, the account holder generates a public/private key pair. The private key is kept secret and used to sign transaction data; the corresponding public key is registered with the financial institution and used by anyone to verify a signature. Mathematically, it is computationally infeasible to derive the private key from the public key, or to forge a valid signature without possessing the private key (this relies on hard problems such as integer factorization for RSA or the elliptic-curve discrete logarithm problem for ECDSA). Consequently, a valid signature on a transaction is normally treated as conclusive cryptographic proof that the registered key holder approved it — this is precisely what authentication and non-repudiation mean in this context.

2.3 Impact of Private Key Compromise

2.3.1 Effect on Authentication

Authentication is completely and immediately broken once the private key is stolen. Since the verification process only checks whether a signature was produced by the key mathematically corresponding to the registered public key — not who physically performed the signing — the system has no way to distinguish a signature produced by the legitimate user from one produced by the attacker now holding the same key. Every unauthorized transaction signed by the attacker will pass verification exactly as if it had been signed by the genuine account holder, so the system's authentication mechanism, from a purely technical standpoint, is fully deceived; it correctly verifies the cryptographic

signature but that verification no longer implies the intended real-world fact (that the legitimate user approved the transaction).

2.3.2 Effect on Non-repudiation

Non-repudiation is undermined in a more subtle and legally significant way. Under normal assumptions, a valid signature is treated as unforgeable proof binding the transaction to the account holder, meaning the account holder cannot later deny signing it. Once a private key has been stolen, this assumption is invalidated: the legitimate user can now plausibly claim — truthfully — that they did not sign the fraudulent transactions, because the signature was in fact produced by an attacker. This creates a genuine dispute resolution problem for the institution: cryptographically, both the legitimate transactions and the attacker's fraudulent ones bear equally “valid” signatures from the same key, so the signature alone can no longer reliably establish who performed any specific transaction. This is precisely why real-world non-repudiation depends not only on signature validity but also on properly protecting the key throughout its lifecycle and, ideally, on supplementary evidence (timestamps, device fingerprints, out-of-band confirmation) that can corroborate or contest a disputed signature after the fact.

2.4 Broader Consequences

- Financial loss: Unauthorized transactions can be executed and potentially settled before the compromise is detected, causing direct monetary loss to the account holder and reputational/regulatory exposure for the institution.
- Legal and regulatory disputes: Because signatures from the compromised key can no longer be trusted to establish who acted, resolving disputed transactions may require costly investigation, forensic analysis, and potentially litigation, and may trigger regulatory reporting obligations (e.g., under RBI/PCI-DSS/GDPR-style breach-notification regimes depending on jurisdiction).
- Erosion of trust in the PKI: If the institution cannot show that private keys were adequately protected (e.g., via a Certificate Authority audit trail, HSM usage, or key-escrow logs), the credibility of its entire digital-signature-based approval system is called into question, not just the one compromised account.
- Need for key revocation: The compromised public key certificate must be revoked (via a Certificate Revocation List or OCSP) and a new key pair issued, and every relying party's trust store must be updated promptly, or the attacker retains the ability to keep signing transactions.

2.5 Mitigation Strategies

2.5.1 Protecting Private Keys (Preventive)

Strategy	Description
----------	-------------

Hardware Security Modules (HSM) / Smart Cards	Generate and store private keys inside tamper-resistant hardware so the raw key material never leaves the device and is never exposed to the host operating system or malware
Multi-factor key release	Require a PIN, biometric, or one-time code in addition to physical possession of the key device before a signing operation can be performed
Threshold / multi-signature schemes	Require signatures from multiple independent key shares (e.g., 2-of-3 threshold ECDSA, or multi-sig approval for high-value transactions) so a single stolen key is insufficient to authorize a transaction
Short-lived session keys / re-authentication	Avoid keeping the private key “unlocked” in memory for long periods; require fresh authentication for each signing session or high-value transaction
Endpoint security	Enforce anti-malware, disk encryption, and OS hardening on any device that has access to the key material or signing application, since key theft frequently occurs via compromised endpoints rather than direct attacks on the cryptography
Key lifecycle management	Enforce periodic key rotation, secure key generation ceremonies, and strict access-control/audit logging over who can request signing operations

Table 2: Preventive strategies to secure private signing keys.

2.5.2 Detective and Responsive Measures

- Anomaly detection / behavioural monitoring: Flag transactions that deviate from a user's normal pattern (unusual amount, beneficiary, device, location, or time) for additional verification even though the signature itself is cryptographically valid.
- Transaction limits and step-up authentication: Require an additional out-of-band confirmation (e.g., SMS/app push approval) for transactions above a risk threshold, so a stolen key alone is insufficient for large unauthorized transfers.
- Rapid key revocation process: Maintain a fast, well-tested Certificate Revocation List / OCSP responder update procedure so a reported compromise can be propagated to all relying parties within minutes, not days.
- Immutable audit logging: Log all signing requests, IP addresses, device identifiers, and timestamps in a tamper-evident log (e.g., append-only or blockchain-anchored) to support forensic reconstruction of events and strengthen the institution's evidentiary position in any dispute.
- Cyber-insurance and incident-response planning: Maintain a rehearsed incident-response plan and appropriate insurance coverage to limit financial and reputational damage once a compromise is confirmed.

2.5.3 Architectural Improvement: Moving Beyond a Single Static Key

Beyond protecting a single private key more strongly, the underlying architecture can be redesigned so that no single stolen key is ever sufficient to authorize a high-value

transaction. Threshold signature schemes distribute the signing capability across multiple independent parties or devices (for example, the user's phone, a bank-held server, and a backup recovery share), such that a valid signature can only be produced when a quorum of these shares cooperates. This means an attacker who compromises only one share — e.g., the user's device — still cannot forge a valid transaction signature, converting a single point of failure into a distributed trust model. This is the same principle used in modern institutional cryptocurrency custody and is increasingly applied to traditional financial transaction-approval systems.

2.6 Illustrative Pseudocode: Step-Up Verification Before Honoring a Signature

```
def process_transaction(tx, signature, public_key):
    # Step 1: standard cryptographic verification
    if not verify_signature(tx, signature, public_key):
        reject(tx, reason="Invalid signature")
    return

    # Step 2: risk-based anomaly scoring even though signature is valid
    risk_score = evaluate_risk(
        amount=tx.amount,
        beneficiary=tx.beneficiary,
        device_fingerprint=tx.device_id,
        geolocation=tx.geo,
        user_history=get_user_profile(tx.user_id)
    )

    # Step 3: require step-up (out-of-band) confirmation for high-risk actions
    if risk_score > RISK_THRESHOLD or tx.amount > HIGH_VALUE_LIMIT:
        if not request_out_of_band_confirmation(tx.user_id, tx):
            reject(tx, reason="Step-up authentication not confirmed")
    return

    # Step 4: log immutably before committing
    audit_log.append_tamper_evident(tx, signature, risk_score)
    commit_transaction(tx)
```

This pattern illustrates a defence-in-depth principle: a valid digital signature is treated as necessary but not always sufficient for high-risk actions, so a stolen key cannot, by itself, guarantee an attacker's transaction will be honoured.

2.7 Conclusion

The theft of a private signing key defeats the fundamental assumption underlying digital signatures: it fully breaks authentication, since the system can no longer distinguish genuine transactions from attacker-forged ones bearing an equally valid signature, and it seriously undermines non-repudiation, since the legitimate user can now truthfully deny having signed the fraudulent transactions, leaving the institution unable to rely on the signature alone to resolve the dispute. Mitigating this risk requires a defence-in-depth approach: hardening key storage with HSMs and multi-factor key release, moving toward threshold/multi-signature architectures that eliminate any single point of key-compromise

failure, layering anomaly detection and step-up authentication on top of cryptographic verification, and maintaining rapid key-revocation and tamper-evident audit capabilities so that when a compromise does occur, it can be detected, contained, and cryptographically and forensically resolved as quickly as possible.